D I S T

Distribuzioni non uniformi

Dai tempi di attesa alle misure con errore: esponenziale e gaussiana

Shahram Rahatlou

Corso di Laboratorio di Calcolo, Canale Q-Z
Anno Accademico 2026-2027

SAPIENZA
UNIVERSITÀ DI ROMA

Eventi che arrivano a caso

Quanto devo aspettare il prossimo? Esempi di tempi di attesa

Telefonate

quanto aspetto la prossima chiamata al centralino? Anche clienti allo sportello o messaggi in un gruppo

Una lampadina

dopo quanto tempo si brucia? Se non si consuma, ogni secondo ha la stessa probabilità di rompersi

Decadimento radioattivo

dopo quanto tempo un nucleo instabile decade? Non «invecchia»: vale ogni istante

Raggi cosmici

tempo tra due particelle che attraversano un rivelatore, o tra due scatti di un contatore Geiger

Terremoti

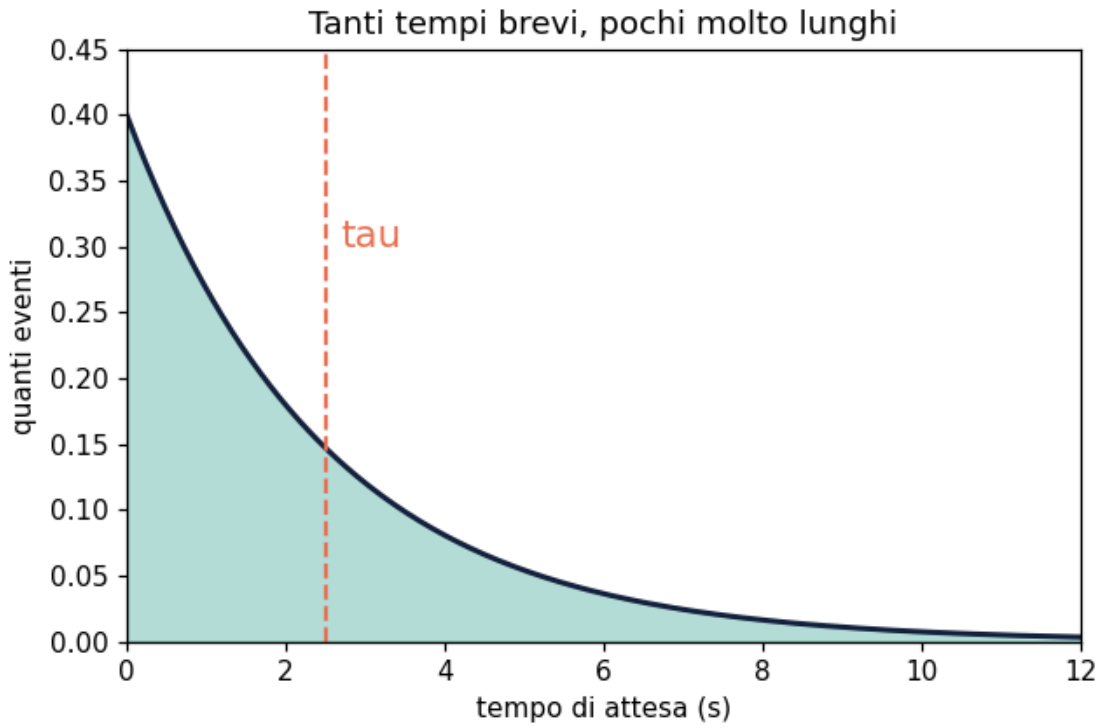
tempo tra due scosse in una stessa zona; vale anche per le stelle cadenti

Cosa hanno in comune?

Ogni istante ha la stessa probabilità che l'evento accada: nessuna memoria. Il tempo di attesa ha sempre la stessa forma.

Non tutti i valori sono ugualmente probabili

Il tempo di attesa tra due eventi: tanti brevi, pochi molto lunghi



Tempo medio tau

il tempo medio di attesa: qui tau = 2.5 s

Legge esponenziale

$$f(t) = \frac{1}{\tau} e^{-t/\tau}$$

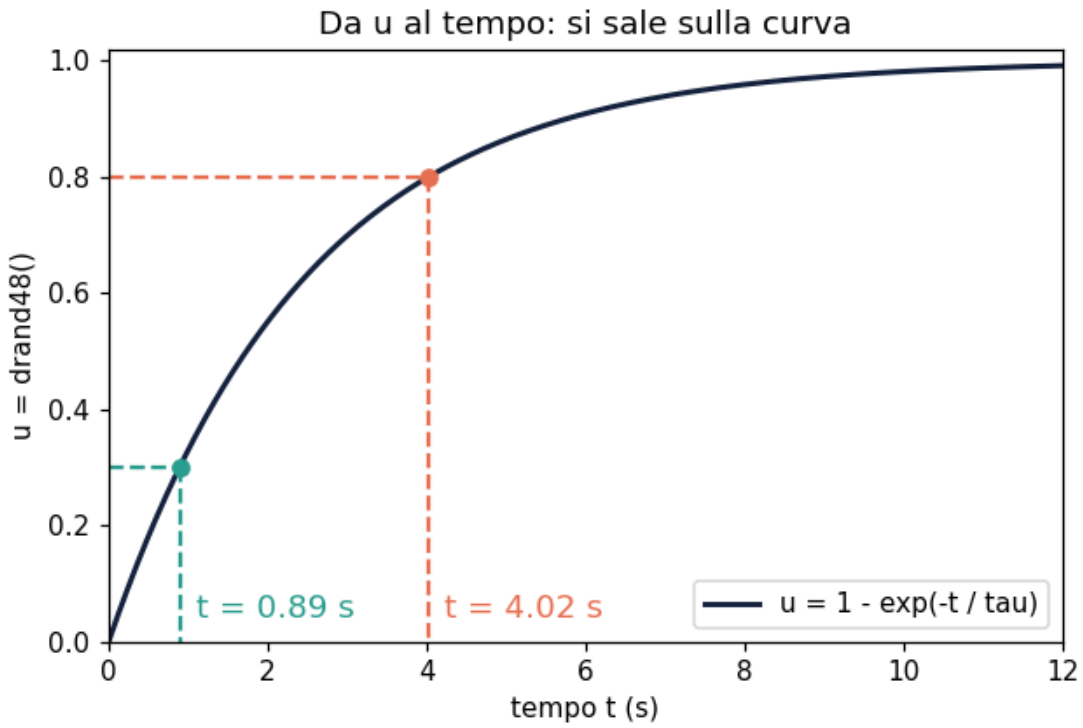
più alta all'inizio, poi sempre più bassa

La domanda

drand48() dà numeri uniformi in [0, 1): come si ottengono tempi con questa forma?

Dall'uniforme all'esponenziale

Si sceglie u sull'asse verticale e si risale al tempo t sulla curva




```
u = drand48();
t = -TAU * log(1. - u);
```

Molti u, pochi t

dove la curva è ripida cadono molti u: molti tempi brevi

Perché 1. - u

u può essere 0 ma 1 - u mai: log(0) non esiste

log di math.h

il logaritmo naturale; si compila con -lm

Tempi di attesa in C

esponenziale.c: 100000 tempi su file, la media e la frazione oltre tau

esponenziale.c (le parti nuove)

```c
#define N 100000
#define TAU 2.5      // tempo medio di attesa (s)

int main() {
    FILE *fp;
    double u, t, somma = 0.;
    int i, oltre = 0;
    // srand48 e fopen come prima
    for (i = 0; i < N; i++) {
        u = drand48();
        t = -TAU * log(1. - u);    // l'esponenziale
        fprintf(fp, "%.6f\n", t);
        somma += t;
        if (t > TAU) {
            oltre++;
        }
    }
    fclose(fp);
    printf("media dei tempi: %.4f\n", somma / N);
    printf("frazione oltre tau: %.4f\n", (double) oltre / N);
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe esponenziale.c -lm
$ ./app.exe
media dei tempi: 2.5031
frazione oltre tau: 0.3675
$ wc -l arrivi.txt
  100000 arrivi.txt
```

La media è tau

2.5031 contro tau = 2.5 s

0.3675 = 1/e

frazione delle attese più lunghe del tempo medio

L'istogramma conferma la formula

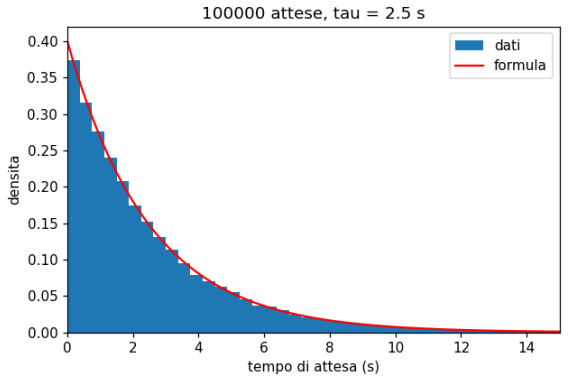
density=True: l'area è 1 e si può confrontare con la curva teorica

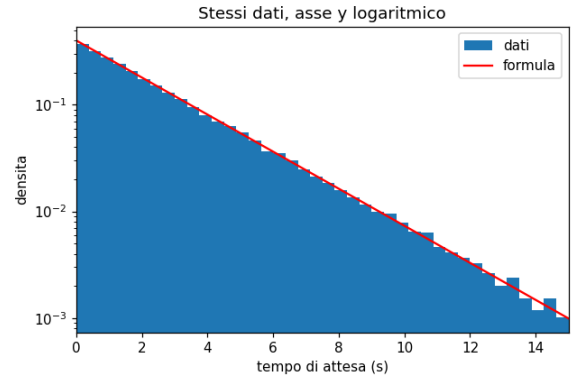
esponenziale.py

```
import numpy as np
import matplotlib.pyplot as plt

TAU = 2.5
t = np.loadtxt('arrivi.txt')

plt.hist(t, bins=40, range=(0., 15.),
density=True, label='dati')
tt = np.linspace(0., 15., 200)
f = np.exp(-tt / TAU) / TAU
plt.plot(tt, f, 'r-', label='formula')
plt.xlim(0., 15.)
plt.xlabel('tempo di attesa (s)')
plt.ylabel('densita')
plt.title('100000 attese, tau = 2.5 s')
plt.legend()
plt.savefig('esponenziale.png')
plt.show()
```





density=True

le barre diventano densità: area totale 1

range

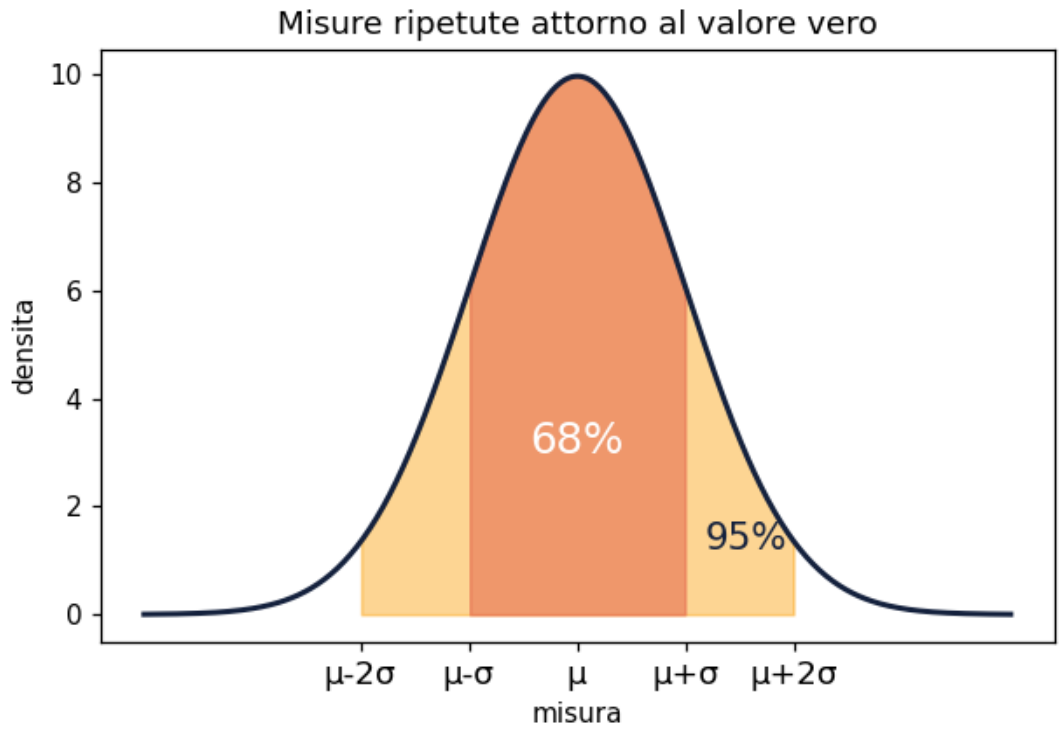
intervalli solo tra 0 e 15 s

yscale('log')

con l'asse logaritmico diventa una retta

Misure con errore: la gaussiana

Ripetendo una misura i valori si addensano attorno al valore vero



mu e sigma

mu è il valore vero (qui 25.00 mm), sigma l'errore della misura (0.04 mm)

68% entro 1 sigma

circa 2 misure su 3 cadono tra mu - sigma e mu + sigma

95% entro 2 sigma

quasi tutte le misure: solo 1 su 20 ne è fuori

Dall'uniforme alla gaussiana

Due numeri uniformi producono un numero gaussiano: una ricetta, senza dimostrazione

la ricetta (Box-Muller)

```
u1 = drand48();
u2 = drand48();
z = sqrt(-2. * log(1. - u1)) * cos(2. * M_PI * u2);
x = MU + SIGMA * z;
```

z

gaussiana standard: media 0 e sigma 1

x = MU + SIGMA * z

sposta il centro in MU e allarga la campana di SIGMA

Due uniformi

u1 e u2 sono due chiamate diverse di drand48(): servono entrambe

math.h

sqrt, log, cos e la costante M_PI: si compila con -lm

Si verifica

con l'istogramma e con le percentuali 68% e 95%

Misure simulate in C

gaussiana.c: 100000 misure su file e le percentuali entro 1 e 2 sigma

gaussiana.c (le parti nuove)

```c
#define MU 25.00      // valore vero (mm)
#define SIGMA 0.04    // errore della misura (mm)
int main() {
    double u1, u2, z, x;
    int i, entro1 = 0, entro2 = 0;
    // srand48 e fopen come prima
    for (i = 0; i < N; i++) {
        u1 = drand48();
        u2 = drand48();
        z = sqrt(-2. * log(1. - u1)) * cos(2. * M_PI * u2);
        x = MU + SIGMA * z;
        fprintf(fp, "%.6f\n", x);
        if (fabs(x - MU) < SIGMA) {
            entro1++;
        }
        if (fabs(x - MU) < 2. * SIGMA) {
            entro2++;
        }
    }
    printf("entro 1 sigma: %.1f%%\n", 100. * entro1 / N);
    printf("entro 2 sigma: %.1f%%\n", 100. * entro2 / N);
```

Terminale

```
$ gcc -Wall -Wextra -o app.exe gaussiana.c -lm
$ ./app.exe
entro 1 sigma: 68.4%
entro 2 sigma: 95.4%
```

68.4% entro 1 sigma

attesi 68%: circa 2 misure su 3

95.4% entro 2 sigma

attesi 95%: quasi tutte le misure

L'istogramma è una campana

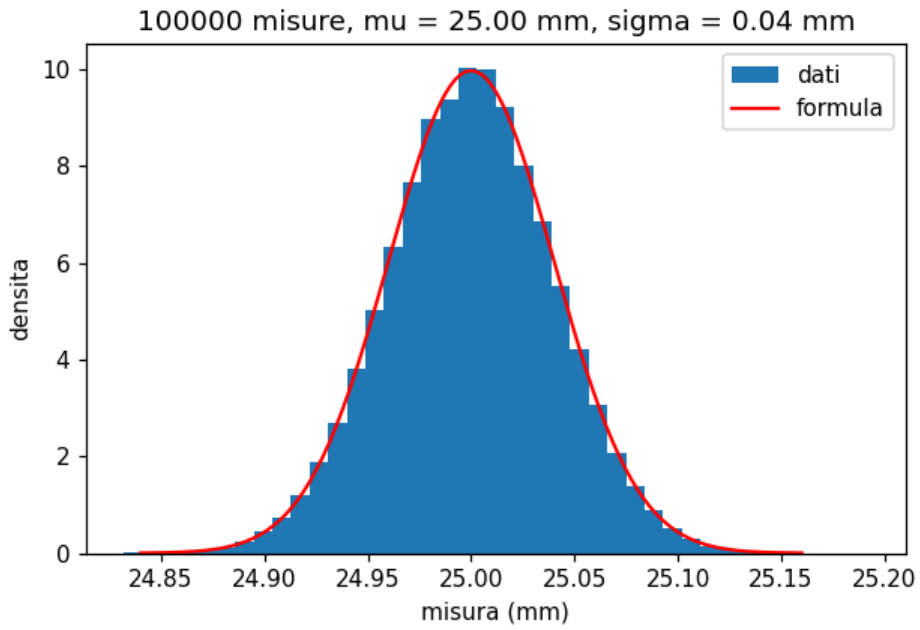
Gli stessi comandi di prima, con la formula della gaussiana sopra le barre

gaussiana.py

```
import numpy as np
import matplotlib.pyplot as plt

MU = 25.00
SIGMA = 0.04
x = np.loadtxt('misure.txt')

plt.hist(x, bins=40, density=True, label='dati')
xx = np.linspace(MU - 4. * SIGMA, MU + 4. * SIGMA, 200)
f = np.exp(-(xx - MU)**2 / (2. * SIGMA**2))
f = f / (SIGMA * np.sqrt(2. * np.pi))
plt.plot(xx, f, 'r-', label='formula')
plt.xlabel('misura (mm)')
plt.ylabel('densita')
plt.title('100000 misure, mu = 25.00 mm, sigma = 0.04 mm')
plt.legend()
plt.savefig('gaussiana.png')
plt.show()
```



np.sqrt, np.pi

dalla libreria numpy

Dati e formula

le barre seguono la curva

Tutte le ricette

u, u1 e u2 sono numeri estratti con drand48(); MU, SIGMA e TAU sono costanti

cosa si vuole	come si scrive
reale in [0, 1)	`drand48()`
reale in [a, b)	`a + (b - a) * drand48()`
intero in [n, m]	`n + (int)(drand48() * (m - n + 1))`
esponenziale, tempo medio TAU	`-TAU * log(1. - u)`
gaussiana standard (z)	`sqrt(-2. * log(1. - u1)) * cos(2. * M_PI * u2)`
gaussiana, media MU e sigma SIGMA	`MU + SIGMA * z`

Per log, sqrt, cos e M_PI: `#include <math.h>` e compilare con `-lm`

Esercizi

Da fare a casa, poi si controlla con un programma in C

1

Attese

Un autobus passa in media ogni 8 minuti, senza orario: le attese sono esponenziali con tau = 8. Simula 20000 attese e scrivile in attese.txt, una per riga. Stampa la media e la frazione di attese più lunghe di 16 minuti. Con Python disegna l'istogramma con l'asse y logaritmico.

per controllare:

```
media vicina a 8
frazione oltre 16: circa 0.135
grafico: una retta
```

2

Altezze

Le altezze di un gruppo di persone hanno media 170 cm e sigma 7 cm. Simula 5000 altezze con la ricetta di Box-Muller e scrivile in altezze.txt, una per riga. Stampa la percentuale di persone alte tra 163 e 177 cm (entro un sigma). Con Python disegna l'istogramma.

per controllare:

```
percentuale vicina a 68%
grafico: una campana
```

3

Segnale

Un sensore misura y = 2x + 1 con un errore casuale gaussiano di sigma 0.5. Per x da 0 a 5 con passo 0.1 scrivi in segnale.txt due colonne, x e y (51 righe). Con Python disegna i punti misurati e, sopra, la retta y = 2x + 1.

per controllare:

```
y(0) vicino a 1
y(5) vicino a 11
punti attorno alla retta
```

Compila con `gcc -Wall -Wextra -o app.exe nome.c -lm` (log, sqrt e cos vogliono -lm)

Riepilogo

1	**Esponenziale**	t = -TAU * log(1. - u): tempi di attesa con media TAU; media e frazione oltre TAU come controllo
2	**Gaussiana**	x = MU + SIGMA * z, con z di Box-Muller da due numeri uniformi; 68% entro 1 sigma e 95% entro 2
3	**Verificare**	istogramma con density=True e la formula teorica sovrapposta; scala logaritmica per l'esponenziale